

Eco-Loop Building Agents

Hackathon Problem Statement — Question 1

Problem Background

Buildings consume approximately 40% of global energy and remain a primary driver of carbon emissions. Traditional Building Management Systems (BMS) rely on rigid, rule-based schedules that fail to adapt dynamically to real-time changes in weather, occupancy, and grid demands. The integration of AI offers a paradigm shift. By pairing physics-based energy simulation engines with open-source LLMs and standardized communication protocols — like the Model Context Protocol (MCP) — we can create truly autonomous structures. This approach transforms a building from a passive energy consumer into an active, self-correcting agent capable of continuous, real-time optimization.

Technical Core Requirements

1. The Simulation Engine (EnergyPlus)

- Utilize EnergyPlus to run high-fidelity building energy simulations.
- You may use functional libraries (e.g., eppy, PyEnergyPlus, or EMS/BCVTB) to bridge Python or execution runtimes with the underlying Input Data File (.idf) or Functional Mock-up Units (.fmu).

2. The Cognitive Engine & Protocol (OSS LLM & MCP)

- Deploy any modern Open-Source LLM (e.g., Llama 3, Mistral, Qwen) running locally or via a self-hosted API.
- Implement an MCP Server or custom agentic tools. The LLM must use these tools to parse files, extract runtime errors, and execute tasks without human code modification.

3. Closed-Loop Execution Framework

- Feedback (EnergyPlus → AI): The simulation must stream continuous performance metrics (e.g., zone temperatures, indoor air quality, energy consumption, Predicted Mean Vote (PMV) thermal comfort indices).
- Reasoning: The LLM evaluates this data against predefined targets like occupancy comfort, peak demand thresholds, and local carbon grid intensity.
- Control Actions (AI → EnergyPlus): The LLM calculates optimal Energy Conservation Measures (ECMs) and updates dynamic set-points.
- Forward Injection: The computed set-points and supervisory overrides must automatically feed directly back into the active EnergyPlus instance.

Hackathon Objective

You must build a live, operational Physical AI Proof-of-Concept (PoC) that automates smart building operations through an autonomous closed-loop control pipeline. Using EnergyPlus as the digital building sandbox and an open-source LLM (or an MCP server configuration) as the brain, you will construct a dynamic feedback loop. The AI model must ingest real-time sensor data from the simulation, evaluate

variables, and continuously inject forward control actions back into EnergyPlus to prove quantifiable energy and cost savings.

Deliverables

Submit a GitHub repository containing:

1. Fully Functional Source Code: A unified codebase (Python preferred) managing the EnergyPlus API wrapper, the LLM agent orchestration logic, and the communication bus.
2. Building Models (.idf files): The base baseline building file along with the modified versions generated during runtime evaluation.
3. Quantitative Savings Dashboard: A visual dashboard or final data export comparing the baseline operation against your AI-driven closed-loop strategy. You must explicitly prove percentage reductions in total kWh consumed while maintaining thermal comfort boundaries.
4. System Architecture Document: A short Markdown report explaining your tool-calling architecture, prompt engineering strategies, prompt latency management, and your technical approach to handling lengthy simulation logs.
5. PoC Demonstration Video: A maximum 3-minute video recording showing the loop in action — highlighting data transferring live from EnergyPlus to the LLM and the subsequent control actions updating the model parameters automatically.
6. Presentation: A completed presentation using the provided template with relevant slides filled in.

Note: All submission files uploaded must be in PDF or ZIP formats only.

Evaluation Criteria

Criterion	Weight	Description
System Integration	30%	Robustness and reliability of the closed-loop pipeline execution over an extended simulation time horizon without crashing.
Energy Efficiency Realized	25%	Net reduction in energy use achieved by the autonomous agent compared to standard baseline scheduling.
Thermal Comfort & Constraints	20%	Balancing energy savings with human occupant comfort.
Agentic Autonomy & Code Elegance	15%	Effective leverage of open-source LLM tool-calling capabilities, MCP protocols, and self-correction loops.
Presentation & Documentation	10%	Clarity of system architecture design, data visualizations, and overall project delivery.